

Streaming Architecture on AWS

Days 4-6 built the batch lake. Today streaming events arrive: cancellations, refunds, boarding-pass scans. Kinesis primitives (shards, partition keys, sequence numbers, iterators), hot-shard math, producer/consumer contracts. The STREAM CONTEXT block (.windsurfrules → v4) enters.

Day 6 → Day 7

Yesterday closed the books on batch reconciliation. Today the pipeline gets stream-shaped events.

YESTERDAY 5 reconciliation queries against curated.settlement ↓ joined by ↓ TODAY Kinesis stream design — events from cancellations land in the same lake.	YESTERDAY Athena workgroup with cost discipline ↓ unchanged ↓ TODAY Today's queries still work; events from streams flow into curated/ via D8 consumer.	YESTERDAY CATALOG CONTEXT block in .windsurfrules v3 ↓ joined by ↓ TODAY STREAM CONTEXT block in .windsurfrules v4 — stream name, shard count, PK strategy, idempotency-store.
YESTERDAY 6 dims HITL meta-checklist ↓ unchanged ↓ TODAY 7 dims through Query Cost Discipline. Day 7 design discipline still uses the same 7.	YESTERDAY Lambda count: 6 of 10 ↓ unchanged ↓ TODAY 0 new Lambdas today. Day 8 adds producer + THIN consumer + bedrock-enrich (count → 9 of 10).	YESTERDAY Workgroup + SNS alarm preserved ↓ reused ↓ TODAY Day 8 will emit Kinesis CW metrics; Day 10 wires the dashboard. SNS topic ready.

Why streaming now — events shouldn't wait for tomorrow's batch

THE PROBLEM — BATCH-ONLY MISSES URGENCY

A passenger cancels at 14:45 today. With batch-only, the cancellation lands in tomorrow's ingest cycle. Refund delays follow; customer service gets the angry call.

Some events need real-time treatment: cancellations, fare-class changes, security alerts. Streaming is the right tool for these.

But streaming has its own failure modes — hot shards, lost sequence numbers, producer back-pressure, consumer lag. The design today prevents those before Day 8 writes the code.

THE PAYOFF — STREAM PRIMITIVES + CONTRACTS

By 03:30 today you'll have provisioned a 2-shard Kinesis stream, designed the producer + consumer contracts, and committed the STREAM CONTEXT block (.windsurfrules → v4). No code yet — the design is the deliverable.

Day 8 wires the Cascade-authored producer + consumer Lambdas onto this stream. Same lake; events land via Kinesis instead of S3 → SQS. Idempotency now keys off `sequence_number`, not `file_key+line_no`.

By the end of Day 7 you can...

DECIDE

Choose batch vs stream per use case using a 5-question rubric (cadence, latency, joins, audit, schema stability).

CONTRACT

Design producer + consumer contracts as JSON schemas; specify idempotency on sequence_number, error-routing, ReportBatchItemFailures.

PRIMITIVE

Explain Kinesis primitives — stream / shard / record / partition key / sequence number / iterator — and how each maps to throughput, ordering, and replay.

LAYER

Author the STREAM CONTEXT block (.windsurfrules → v4) (stream name, shard count, PK strategy, consumer batch size, idempotency-store).

PARTITION

Avoid hot shards: choose a partition key with high cardinality and even distribution. Apply hot-shard math (1MB/s, 1000 records/s per shard write quota).

PROVISION

Cascade-author the CLI to provision: 1 Kinesis stream (2 shards), 1 IAM role, 1 DynamoDB stream-land idempotency table. Verify with describe-stream.

Four parts. Concepts → Design → Labs → Wrap.

01

Concept briefing

Seven concepts. Batch vs stream. Kinesis primitives. Hot-shard math. Producer + consumer contracts. Sequence-number idempotency. Kinesis vs MSK vs EventBridge. The STREAM CONTEXT block in .windsurfrules v4.

TIME

~35 min

02

Guided design

Four guided slides. Booking-event schema. Producer + consumer JSON contracts. Idempotency table for streams. Hot-shard avoidance with PK = airline_code+booking_date.

TIME

~40 min

03

Three labs

Lab 1 — provision 2-shard stream + IAM via Cascade-authored CLI (45 min). Lab 2 — design producer Lambda contract on paper (45 min). Lab 3 — hot-shard demo: same data, two PK strategies, observe shard distribution (45 min).

TIME

~135 min

04

Review & wrap

Five Day-7 mistakes. Cleanup respects stream state. Day-7 in numbers. Bridge to Day 8 — Cascade-authors the producer + consumer Lambdas. STATE CONTEXT block (.windsurfrules → v5).

TIME

~25 min

01

PART 1 · ~35 MINUTES

Concept briefing

Seven concepts that make the difference between a Kinesis stream that scales and one that wedges. Batch vs stream. Primitives. Hot-shard math. Contracts. Sequence-number idempotency. Kinesis vs MSK vs EventBridge. The .windsurfrules v4 with STREAM CONTEXT.

Same 5-question rubric. Different answer for cancellations vs settlement.

Settlement is end-of-day batch (Day 5/6). Cancellations are real-time stream (Day 7+). The decision uses the same 5 questions as Day 6 but lands on the opposite side.

5-question rubric — when streaming wins

1. CADENCE — sub-second per event? STREAM. Hourly+? BATCH.
2. LATENCY BUDGET — pages-need-now? STREAM. Reconciliation-tomorrow? BATCH.
3. STATE WINDOW — last 5-min rolling? STREAM. Full BSP cycle? BATCH.
4. AUDIT — every event individually traced? STREAM. Once-per-cycle? BATCH.
5. SCHEMA STABILITY — both fine; streams more sensitive to drift.

AIRLINE EVENTS THAT ARE STREAMS:

cancellation, refund, fare-class change, boarding-pass scan, no-show, upgrade, seat-block release, security-alert.

AIRLINE THINGS THAT ARE BATCH:

BSP settlement file (weekly), end-of-cycle reconciliation, monthly remittance, fare table refresh (nightly).

Programme commitment: cancellation events land via Kinesis from Day 8 forward. Settlement stays batch (Day 5 design preserved).

01

Cadence first

Sub-second → stream. Hourly+ → batch.

02

Latency budget

Pages-need-now → stream. Reconciliation-tomorrow → batch.

03

State window

Bounded recent → stream. Cycle-spanning → batch.

04

Cancellations = stream

Real-time customer impact. Day 8 implements.

05

Settlement = batch

Day 5/6 preserved unchanged.

Stream / shard / record / partition key / sequence number / iterator.

Five terms cover ~90% of Kinesis. Get them right; everything else is plumbing.

Five primitives + how each maps to a real concern

STREAM	named container; lives in one region e.g., accelya-bookings-stream-<learner>
SHARD	unit of throughput. Per-shard quota: WRITE: 1 MB/s OR 1,000 records/s (whichever first) READ: 2 MB/s OR 5 transactions/s (default; enhanced fan-out higher)
RECORD	partition_key + data + sequence_number Producer sets partition_key + data; AWS sets sequence_number
PARTITION KEY	up to 256 chars. Determines which SHARD the record lands on. hash(partition_key) → range → shard mapping Pick wrong → hot shard. Pick right → even distribution.
SEQUENCE NUMBER	~56-digit string assigned per record per shard. Strictly increasing per shard. THE idempotency key for streams.
ITERATOR	consumer position pointer; types: TRIM_HORIZON, LATEST, AT_SEQUENCE_NUMBER, AFTER_SEQUENCE_NUMBER, AT_TIMESTAMP. ESM uses TRIM_HORIZON for first read, then resumes from last-processed sequence.

01

Shard = throughput unit

1MB/s OR 1000 rec/s write. Plan shard count for peak.

02

PK → shard

hash(PK) decides shard. Hot PK = hot shard.

03

Sequence number

Strictly increasing per shard. THE idempotency key.

04

Iterator types

TRIM_HORIZON for replay; LATEST for live-only; AT/AFTER for resume.

05

ESM auto-resumes

Lambda event-source-mapping persists last sequence; restarts from there.

Wrong PK → 1 shard does all the work; the rest sit idle.

Quick math: a 2-shard stream gives you 2MB/s and 2000 rec/s of write. If 95% of records hash to one shard, you have 1MB/s + 1000 rec/s — same as a 1-shard stream and you pay double. Pick the PK on cardinality and distribution.

PK choices for the cancellation stream — worked through

```
BAD: PK = booking_id (UUID-like, ~365 days × 100K bookings = 36.5M unique IDs)
Distribution: even ✓ (random UUIDs hash uniformly)
But: NO ordering guarantees — same booking's events scatter across shards.
Refund-after-cancel may arrive BEFORE cancel; consumer can't reason about state.

BAD: PK = airline_code (4-10 distinct values)
Distribution: skewed ✗ (BSP airlines hit 1 shard; ATPCO hits another)
At 4 airlines + 2 shards: 50% of traffic on the busy shard.

BAD: PK = static 'cancellation' string
Distribution: 100% on one shard. ✗ This is a 1-shard stream in disguise.

BASELINE (teaching): PK = airline_code + ':' + booking_date
~1460 unique PKs/year; even hash. BUT: in production, today's traffic is
concentrated on TODAY's (airline, date). At burst, that single key dominates one
shard regardless of annual cardinality. Use this only as the teaching baseline.

RECOMMENDED (high-volume, ORDER-TOLERANT events):
airline_code+':'+booking_date+':'+str(bucket_hash(pnr) % N)
bucket_hash(pnr) % 8 splits same-airline-same-date traffic across N=8 buckets.
Distributes peak-day burst across multiple shards (~1/N each).
Co-locates events that hash to the same bucket, but does NOT isolate per-PNR.
All PNRs in the same bucket SHARE one key — ordering is within bucket, not per-PNR.

RECOMMENDED FOR STRICT PER-PNR CHAINS (cancel→refund): airline_code+':'+booking_date+':'+pnr_hash
Full partition key includes the PNR (or a deterministic hash of it).
Same PNR = same key = same shard = strict per-PNR ordering.
Higher cardinality. PII policy: prefer pnr_hash over raw PNR if PII rules require.
Combine with SequenceNumberForOrdering on PutRecord for AWS-guaranteed ordering.
```

01

Cardinality matters

Need many distinct PKs to spread across shards.

02

Distribution matters

Even hash distribution avoids skew.

03

Ordering matters

Same PK = same shard = order preserved per-PK.

04

Default: airline + date

1460 unique PKs; even; ordering for (airline, date).

05

Per-shard quota: 1MB/s, 1000 rec/s

Plan for peak across shards.

What a well-behaved producer looks like — PutRecord vs PutRecords, deterministic PK, retry.

The producer is half the contract. Get it wrong and the consumer pays.

Producer contract — TWO paths: bulk (PutRecords) and strict-order (PutRecord)

DETERMINISTIC PARTITION KEY

```
pk = f"{airline_code}:{booking_date.isoformat()}:{bucket_hash(pnr) % 8}"
Same booking/PNR → same PK → same shard (per-PK ordering preserved).
Bucket suffix distributes high-volume single days across multiple shards.
```

TWO PRODUCER PATHS — choose per use-case:

(A) BULK PATH — PutRecords ≤500 records

Use for: high-throughput, ORDER-TOLERANT events (e.g. independent cancellations from different bookings).

WARNING: PutRecords does NOT guarantee ordering — even within a single batch for the same PK, AWS may reorder records on the wire.

Inspect FailedRecordCount + per-record error; re-PutRecords just failed.

(B) STRICT-ORDER PATH — sequential PutRecord with SequenceNumberForOrdering

Use for: ordered chains that MUST preserve order:

cancellation → refund for the SAME booking/PNR.

Producer writes record N with SequenceNumberForOrdering=<prev_seq>;

awaits ACK; passes returned SequenceNumber on the NEXT write.

AWS guarantees strictly increasing sequence numbers when this is used.

Same full partition key MUST be identical for every event in the chain.

Slower; reserve for chains that need it.

RETRY ON THROTTLE (ProvisionedThroughputExceededException)

Exponential backoff with jitter; cap 5 retries.

Producer tracks Failed records and re-PutRecords just those.

ORDER GUARANTEES (per-shard only, not global)

Producer assumption: events for (airline, date) arrive at consumer in order.

AWS guarantees per-shard order. Producer + PK choice make it work.

01

Deterministic PK

Same logical event → same PK → same shard.

02

PutRecords preferred

Batch of 500. Inspect FailedRecordCount.

03

Throttle retry

Exponential backoff + jitter; cap 5; re-PutRecords just failed.

04

Per-shard ordering only

Global ordering = anti-pattern. Plan for it.

05

EMF metrics

records_in, records_failed, retry_count tagged per stream + airline.

ESM + ReportBatchItemFailures + sequence-number idempotency.

The consumer is the other half. Day 8 writes it; today you design it.

Consumer contract — idempotent, partial-batch-aware, observable

EVENT-SOURCE MAPPING (ESM)

Lambda triggered per shard (default 1 invocation per shard at a time).
 BatchSize: up to 10000 records per invoke (default 100).
 ParallelizationFactor: 1-10 invocations per shard (1 = ordering preserved).
 StartingPosition: TRIM_HORIZON (replay) or LATEST (live-only).
 FunctionResponseType: [ReportBatchItemFailures] # CRITICAL.

IDEMPOTENCY ON SEQUENCE NUMBER

pk = f'stream#{stream_name}#{shard_id}#{sequence_number}'
 DynamoDB conditional put (same pattern as Day 4).
 Day 8 builds: stream_seen_keys table; Day 10 wires CW alarms on miss rate.

PARTIAL-BATCH FAILURES — KINESIS RBIF differs from SQS

Lambda return: {'batchItemFailures': [{'itemIdentifier': '<sequenceNumber>'}]}.
 CRITICAL: ESM checkpoints at the LOWEST failed sequence_number and retries EVERYTHING from that checkpoint forward — NOT per-message retry like SQS.
 Successful records that were AFTER the failed seq# get re-delivered.
 Idempotency-claim must catch this (claim before any side-effect).
 Producer rule: report only the OLDEST failure; reporting stale failures wastes work.

BACKPRESSURE

IteratorAge metric is the leading indicator; alarm at 60s lag.
 Day 10 wires the alarm; today the design names the metric.

01

ReportBatchItemFailures

Same as Day 4 SQS. Partial-batch is the contract.

02

PK = stream#shard#seq

Sequence number per shard. Idempotency stable across replays.

03

ParallelizationFactor 1

Per-PK ordering preserved. Bump only if you can tolerate scatter.

04

IteratorAge alarm

Leading indicator of consumer lag. Alarm 60s.

05

ESM resumes oldest-failed

Reports failed seqs; ESM re-reads from there. No skip risk.

When to pick which.

Three streaming services. They overlap on use cases. Sandbox supports all three; today's lab uses Kinesis (cheapest + simplest for the airline use case).

Decision matrix — pick the right tool

AMAZON KINESIS DATA STREAMS

Use when: ordered per-PK; replay needed; <10MB/s/shard fine; pay per shard-hour.

Strengths: simple consumer model (ESM); replay via TRIM_HORIZON; cheap at modest scale.

Weaknesses: shard-level throughput limits; rebalancing requires resharding.

AMAZON MSK (Managed Streaming for Apache Kafka)

Use when: existing Kafka shop; cross-region replication; >10MB/s/partition needed; complex stream processing (KSQL, Kafka Streams, Flink).

Strengths: industry standard; rich ecosystem; long retention.

Weaknesses: more operational overhead; pricier than Kinesis at the same scale.

AMAZON EVENTBRIDGE

Use when: SaaS event integration; many-to-many fan-out; rule-based routing.

Strengths: 100s of source/target integrations; serverless; pay per event.

Weaknesses: not a stream — no replay, no ordering, no per-key partitioning.

DAY 7-8 CHOICE: Kinesis. Reasons:

- Per-PK ordering (airline, date) needed for cancellation event sequencing.
- Replay (TRIM_HORIZON) for late-discovered consumer bugs.
- Cheapest at the lab + production scale we project.
- Simplest consumer model — Lambda + ESM.

01

Kinesis

Ordered per-PK + replay + cheap. Lab default.

02

MSK

Kafka-native + complex stream processing.

03

EventBridge

Many-to-many routing. NOT a stream.

04

Per-shard quota

Kinesis: 1MB/s, 1000 rec/s. MSK: per-partition higher.

05

Sandbox supports all 3

Today: Kinesis only.

Added to .windsurfrules v3 (becomes v4).

Cascade now reads `STREAM CONTEXT` before scaffolding any producer or consumer Lambda.

STREAM CONTEXT layer template

```
## #STREAM CONTEXT
Stream name:      accelya-bookings-stream-<learner>
Shard count:     2 (sandbox lab default; production will likely be 4-8)
Region:          ap-south-1
PK strategy (TWO TIERS):
  Bulk path (order-tolerant, high-volume):
    f'{airline_code}:{booking_date.isoformat()}:{bucket_hash(pnr) % 8}'
    Distributes burst across N=8 buckets per (airline, date).
    NOTE: events in the same bucket share a key – ordering is per-bucket, not per-PNR.
  Strict per-PNR chains (cancel→refund):
    f'{airline_code}:{booking_date.isoformat()}:{pnr_hash}'
    Full key includes pnr_hash → same PNR = same shard = strict per-PNR order.
    Combine with SequenceNumberForOrdering on PutRecord for guaranteed ordering.
ESM config (lab-safe explicit defaults – never rely on AWS defaults):
  FunctionResponseTypes:      [ReportBatchItemFailures]
  ParallelizationFactor:      1 # ordering preserved per shard
  MaximumRetryAttempts:       3 # NOT -1 (retry-until-expiry default)
  MaximumRecordAgeInSeconds:   3600 # 1h NOT -1 (until expiry default)
  BisectBatchOnFunctionError:  true # halve + retry to isolate poison record
  StartingPosition:           TRIM_HORIZON
  BatchSize:                  100
  MaximumBatchingWindowInSeconds: 5
Consumer batch: 100 records per invoke (default)
Idempotency-store:
  Table: stream_seen_keys_<learner>
  PK: f'stream#{stream_name}#{shard_id}#{sequence_number}'
  TTL: 7 days (cancellation events; tune for use case)
Iterator default: TRIM_HORIZON for first read (replay-friendly)
                  ESM persists last-processed sequence per shard
```

01

Layer slot

After BATCH CONTEXT, before CATALOG CONTEXT.

02

Stream + shard + PK

Cascade hardcodes against accelya-bookings-stream + 2 shards.

03

Idempotency on seq#

Day 4's pattern, applied to streams. Day 8 builds the table.

04

Producer rules

PutRecords, deterministic PK, retry+backoff, EMF metrics.

05

Consumer rules

ESM + RBIF + idempotency + Catch routing per Day 4 ladder.

Day 7 · Module 7 · Streaming Architecture on AWS

Streaming vocabulary set — 7 concepts, .windsurfrules v4 with STREAM CONTEXT ready

Batch vs stream rubric. Kinesis primitives. Hot-shard math (PK choice on cardinality + distribution + ordering). Producer + consumer contracts. Sequence-number idempotency. Service comparison. The STREAM CONTEXT block (.windsurfrules → v4).

VOCABULARY

Stream/shard/record/PK/sequence-number/iterator.
PutRecords. ReportBatchItemFailures. STREAM CONTEXT.
Used in every Day-7+ Cascade prompt.

WHAT'S NEW

STREAM CONTEXT block in .windsurfrules v4. Hot-shard math as the design discipline. Sequence number replaces file_key+line_no for idempotency.

WHAT'S NEXT

Four guided design slides: booking-event schema, producer + consumer JSON contracts, idempotency table, PK choice with hot-shard avoidance.

02

PART 2 · ~40 MINUTES

Guided design

Four guided design slides. The booking-event schema (cancellation, refund, no-show, upgrade, seat-block release). Producer + consumer JSON contracts. Stream-idempotency table schema. Hot-shard avoidance with PK = airline_code + ':' + booking_date.

One Kinesis stream carries multiple event types. Discriminator field routes the consumer.

ENVELOPE

Common across all event types. Discriminator: event_type.

```
{
  "event_id":      "evt-2026-W19-AI234-CXL-001",
  # globally unique
  "event_type":    "cancellation",  #
  "cancellation | refund |

                                #
no_show | upgrade | seat_release
  "event_time":    "2026-05-09T14:45:00+05:30",
  "ingest_time":   "2026-05-09T14:45:02Z",
  "airline_code":  "AI",
  "booking_date":  "2026-05-09",
  "pnr":           "6X7Y8Z",
  "source_system": "GDS",
  "payload":       { /* event-type-specific */ }
}
```

PER-TYPE PAYLOAD

Cancellation includes refund hint; refund references the cancellation_id.

```
// cancellation
{ "reason_code": "PASSENGER_REQUEST",
  "channel": "web",
  "refund_eligible": true, "settlement_id": "BSP-
2026-W19-AI234-001" }

// refund (refers to cancellation_id)
{ "cancellation_event_id": "evt-...-CXL-001",
  "refund_amount": 2845.00,
  "currency": "INR",
  "refund_method": "original_card" }

// no_show
{ "check_in_status": "NEVER",
  "flight_number": "AI234", "departure": "..." }

// upgrade
{ "from_class": "Y", "to_class": "J",
  "upgrade_price": 12500.00 }

// seat_release
{ "seat": "14A", "hold_expired_at": "2026-05-
09T14:30:00+05:30" }
```

CONSUMER ROUTING

Switch on event_type. Each handler has its own quarantine and idempotency.

```
def route(record):
    e = json.loads(record['data'])
    et = e['event_type']
    handler = {
        'cancellation': handle_cancellation,
        'refund': handle_refund,
        'no_show': handle_no_show,
        'upgrade': handle_upgrade,
        'seat_release': handle_seat_release,
    }.get(et)
    if handler is None:
        raise ValidationError(f'unknown
event_type {et}'),

    file_key=e.get('event_id'))
    return handler(e)
```


Both contracts as JSON Schemas. Cascade lints against them in Day 8.

PRODUCER CONTRACT

Inputs to the producer Lambda; outputs to Kinesis PutRecords.

```
## Input (per-invocation event)
{ 'events': [ <envelope>, ... ] }

## Output (return value)
{
  'submitted_count': int,
  'failed_count': int,
  'failed_records': [
    { 'event_id': str, 'error_code': str,
      'error_message': str }
  ]
}

## Side effects
PutRecords against accelya-bookings-stream-
<learner>
Each record:
  PartitionKey:
    f'{airline_code}:{booking_date}'
  Data: JSON envelope
  EMF: records_in, records_failed, retry_count
  tagged stream+airline.
```

CONSUMER CONTRACT

Triggered by ESM. Returns batchItemFailures.

```
## Input (from ESM)
{ 'Records': [ {
  'kinesis': {
    'partitionKey': '...',
    'sequenceNumber': '...',
    'data': '<base64-encoded JSON envelope>'
  },
  'eventID': 'shardId-...:sequenceNumber'
} , ... ] }

## Output
{ 'batchItemFailures': [
  { 'itemIdentifier': '<sequenceNumber>' }
] }

## Side effects per record
1. Idempotency claim on
stream#stream_name#shard_id#sequence_number
- dup → silent skip (NOT
batchItemFailures)
2. Schema-validate (event_type + envelope +
payload)
- ValidationError →
quarantine/<learner>/stream_invalid/<reason>/
(NOT
batchItemFailures)
3. Route by event_type → per-type handler
- any unhandled exception →
batchItemFailures (ESM retries)
```

FAILURE → ROUTING

Reuse Day 4 PipelineError ladder. ASL Catch routes to per-type recovery.

```
ValidationError →
quarantine/<learner>/stream_invalid/<reason>/
ManifestMismatch → N/A (no manifests on
streams)
MalformedRowError →
quarantine/<learner>/stream_invalid/parse_error
/
ProvisionedThroughputExceededException
(producer) → retry+backoff (≤5)
TransientError → batchItemFailures + ESM
retry
ConfigError → SNS escalate; do NOT
auto-retry
```

Day 8 provisions; Day 7 designs the schema.

TABLE SCHEMA

PK is the full sequence-key. TTL 7 days for cancellation events.

```
TABLE: stream_seen_keys_<learner>
ATTRIBUTES:
  pk          String  =
'stream#<stream_name>#<shard_id>#<sequence_number>'
  event_id    String  = the
envelope.event_id (audit)
  event_type  String  = cancellation |
refund | ...
  processed_at Number  = epoch seconds
  outcome     String  = 'processed' |
'quarantined' | 'duplicate'
  ttl         Number  = processed_at + 7
days

WRITE PATTERN (same as Day 4 – first-writer-
wins):
  PutItem(pk=key,
ConditionExpression='attribute_not_exists(pk)')
  → ConditionalCheckFailedException → silent
skip
```

WHY 7 DAYS

Cancellation events: replay window narrow. TTL keeps table small.

```
Lifecycle math:
  Peak: ~10K cancellations / hour × 24h × 7d =
~1.7M items at peak retention
  Item size: ~150 bytes ≈ 250MB peak storage
  Storage cost: ~$0.07/learner/month (lab
scale: well under)

Tuning per use-case:
  Settlement events: TTL 30 days (matches Day 3
row-replay TTL)
  Real-time security: TTL 24 hours (don't
replay old security events)
```

ALTERNATIVE: KCL CHECKPOINTING

Out of scope today. Mentioned for awareness.

```
If you used KCL (Kinesis Client Library)
instead of Lambda ESM:
  KCL maintains its own DynamoDB checkpoint
table per stream.
  No need for stream_seen_keys; KCL guarantees
at-least-once.
  Sandbox-acceptable but more operational
overhead.

Today's choice: Lambda ESM + stream_seen_keys
(consistent with Day 4 pattern).
```

Final design check. PK choice locked in. Cascade now hardcodes this in Day 8.

DECISION

PK locked. Cardinality + distribution + ordering all met.

```
PartitionKey:
f"{airline_code}:{booking_date.isoformat()}"

Cardinality: ~1460 unique PKs/year (4 airlines
× 365 days)
Distribution: even hash; ~730 PKs per shard at
2-shard config
Ordering: per-PK preserved (events for same
airline+date land same shard)

Trade-off accepted:
- Same airline + same date → 1 shard. If one
airline-day spikes (mass cancel
after a strike) you can hot one shard
temporarily. Acceptable for lab.
- Production: monitor IncomingBytes per
shard; reshard if sustained imbalance.
```

VERIFICATION (Lab 3 will test)

Two PK strategies sent through the same data; observe distribution.

```
# Lab 3 plan:
# Strategy A: PK = booking_id (random)
- even but no ordering
# Strategy B: PK = airline+date
(deterministic) - even AND ordered
# Same 10K synthetic cancellation events
through each.

# Compare CloudWatch metrics:
# IncomingBytes per shard → both should be
~equal
# IncomingRecords per shard → both should be
~equal
# Then test ordering:
# Cancel-then-refund pair on (AI, 2026-05-
09):
# A (random): ~50% chance refund arrives
BEFORE cancel - bug.
# B (deterministic): always cancel-then-
refund order - correct.
```

OPS NOTE — RESHARDING

Out of scope for the lab. Production reality.

```
If sustained imbalance shows in CW
IncomingBytes per shard:
- Split shard: aws kinesis split-shard --
stream-name ... --shard-to-split ...
- Merge shards: aws kinesis merge-shards --
stream-name ... --shard-to-merge ...
```

Each operation: 5-10 min; existing data not moved (only new writes affected).
Lab today: NEVER reshard; sandbox cap is 2 shards anyway.
Day 10 Observability wires the IncomingBytes alarm; Day 12 capstone may demo.

Streaming design ratified — 4 artefacts ready to provision

Booking-event schema (envelope + 5 payloads). Producer + consumer JSON contracts. Stream-idempotency table. PK locked: airline + date.

EVENT SCHEMA

5 event types share an envelope. event_type is the discriminator. Per-type payloads narrow.

CONTRACTS

Producer: PutRecords + deterministic PK + retry. Consumer: ESM + RBIF + sequence-number idempotency + Catch routing.

PK LOCKED

airline_code+'.'+booking_date. ~1460 unique PKs/year. Per-PK ordering preserved.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 (45 min) — provision 2-shard stream + IAM + DynamoDB stream-idempotency table via Cascade-authored CLI; verify with describe-stream. Lab 2 (45 min) — design producer Lambda contract on paper; commit /design/m7-producer-contract.md. Lab 3 (45 min) — hot-shard demo: send 10K events with two PK strategies; observe shard distribution + ordering.

Provision 2-shard Kinesis stream + IAM + idempotency table

BRIEF

Pair-work. Use prompts/v10_stream.txt (the 10-layer prompt with STREAM CONTEXT). Cascade-author scripts/provision_m7_stream.sh that creates: the Kinesis stream (2 shards, ap-south-1, 24h retention default), the IAM role with stream + DDB scope, and the DynamoDB stream-idempotency table. Verify with describe-stream + describe-table.

TIME

45 min

LEVEL

Working+

DELIVERABLE

prompts/v10_stream.txt (10-layer with STREAM CONTEXT)
scripts/provision_m7_stream.sh
/reviews/m7-provision-evidence.md (describe-stream + describe-table outputs)

Six steps. Branch → v10 prompt → Cascade-author provision script → run → verify stream → verify DDB → commit + MR.

1

Branch + 10-layer prompt template

```
export LEARNER=jai
git checkout main && git pull --ff-only
git checkout -b module-7-$LEARNER
```

2

Cascade-authors provision script

```
# In Windsurf chat: attach prompts/v10_stream.txt
# TASK: scripts/provision_m7_stream.sh per STREAM CONTEXT.
# 1. aws kinesis create-stream --name accelya-bookings-stream $LEARNER --shard-count 2
# 2. wait for ACTIVE
```

3

Run + verify stream

```
bash scripts/provision_m7_stream.sh $LEARNER
aws kinesis describe-stream --stream-name accelya-bookings-stream-$LEARNER \
  --region ap-south-1 \
  --query 'StreamDescription.(Status.StreamStatus.Chards.Length(Chards).Retention.PotentialRetentionPeriodHours)
```

4

Verify IAM + DynamoDB

```
aws iam get-role --role-name accelya-stream-role-$LEARNER \
  --query 'Role.{Name:RoleName,Trust:AssumeRolePolicyDocument}'
aws dynamodb describe-table --table-name stream-seen-keys-$LEARNER \
  --query 'Table.TableName'
```

5

Smoke-test: PutRecord + GetRecords

```
# Manual smoke (not part of producer contract – just proves wiring):
aws kinesis put-record --stream-name accelya-bookings-stream-$LEARNER \
  --partition-key 'AT-2026-05-08' --data 'eyJ3bW9uZSI6dHJ1Y00=' \
  --region us-east-1
```

6

Document + commit + MR

```
# Save the describe-stream/describe-table outputs to /reviews/m7-provision-evidence.md
git add scripts/ prompts/v10_stream.txt reviews/m7-provision-evidence.md
git commit -m "module-7: provision Kinesis stream + IAM + idempotency table (Lab 1)"

git push origin module7:StagingDev $ITERATOR --region ap-south-1 \
  --streaming-architecture-on-aws --partitions-per-key Seq. SequenceNumber,0 --data
```

EXPECTED

On module-7-<learner>. v10_stream.txt has 10 layers; STREAM CONTEXT visible.

EXPECTED

scripts/provision_m7_stream.sh present. Creates stream + IAM role + DDB table.

EXPECTED

Status ACTIVE. Shards: 2. Retention: 24.

EXPECTED

Role exists with stream + DDB scope. DDB Status ACTIVE; TTL Enabled on attribute 'ttl'.

EXPECTED

PutRecord returns SequenceNumber. GetRecords returns 1 record with that PK and base64 data.

EXPECTED

MR open on GitLab; pair partner tagged.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Stream accelya-bookings-stream-<learner> ACTIVE with 2 shards
- DynamoDB stream_seen_keys_<learner> ACTIVE with TTL enabled
- IAM role accelya-stream-role-<learner> with stream + DDB scope
- Smoke-test PutRecord/GetRecords returns matching data
- /reviews/m7-provision-evidence.md captures describe outputs
- MR open with pair partner tagged

⚠ TOP GOTCHAS

Shard count 1

Default is 1; you must pass --shard-count 2. Otherwise hot-shard demo (Lab 3) is meaningless.

Retention < 24h

Default is 24h; for production tune higher (max 8760h = 1 year). Lab keeps default.

IAM trust policy missing

Role must trust lambda.amazonaws.com for Day 8 ESM to work.

TTL on wrong attribute

Must specify AttributeName='ttl' explicitly; DynamoDB doesn't infer.

Cross-region by accident

Always pass --region ap-south-1; otherwise stream lands in us-east-1 default.

Producer contract design — paper exercise

BRIEF

Solo. Use the producer contract template from Guided Design slide 16 (or v10_stream.txt) and write /design/m7-producer-contract.md for the cancellation-event use case.

Cover: input shape, output shape, side effects (PutRecords with PK strategy), retry policy, EMF metrics, observability. NO CODE today — just the design. Day 8 implements.

Pair-review: HITL on the design using 5 dims + the new STREAM dim (per-PK ordering preservation).

TIME

45 min

LEVEL

Working+

DELIVERABLE

/design/m7-producer-contract.md
/reviews/HITL_REVIEW-m7-producer.md (scored 6 dims)

Six steps. Pull contract template → fill cancellation-event use case → write design MD → HITL → commit.

1

Pull contract template

```
# Producer contract template lives in prompts/v10_stream.txt under PRODUCER RULES.
# Copy as starting point:
mkdir -p design
cat prompts/v10_stream.txt | sed -e '/PRODUCER RULES/;/CONSUMER RULES/;/\>/d' > design/m7-producer-contract-
```

EXPECTED

design/m7-producer-contract-WIP.md present. Contains the producer-rules block from v10.

2

Fill cancellation-event specifics

```
# Sections to fill in:
#   ## Input shape
#   event[envelope] with event_type='cancellation' + payload from Guided Design slide 15
#   ## Output shape
```

EXPECTED

All sections present. Specific to cancellation events.

3

Optional: ASCII flow diagram

```
# Useful for the design doc:
[caller]
|
+-- events: {envelope}
#   ## retry policy
```

EXPECTED

Optional but valuable for review.

4

Save to design/m7-producer-contract.md

```
mv design/m7-producer-contract-WIP.md design/m7-producer-contract.md
```

EXPECTED

Final filename in place.

5

Pair-HITL on 6 dims (5 + per-PK ordering)

```
# /reviews/HITL_REVIEW-m7-producer.md scored on:
#   Correctness: schema matches Guided Design slide 15?
#   Safety: IAM scope minimal; no s3:* in producer role?
#   Idempotency: deterministic PK; no random UUID?
```

EXPECTED

Review scored ≥14/18 by both pair members.

6

Commit

```
git add design/m7-producer-contract.md reviews/HITL_REVIEW-m7-producer.md
git commit -m "module-7: producer contract design (Lab 2)"
git push
```

EXPECTED

MR shows 2 commits.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- design/m7-producer-contract.md present with all 6 sections (input/output/side-effects/retry/metrics/failure)
- PK strategy explicit: airline_code+'_'+booking_date
- Retry policy explicit: PTE_x → backoff+jitter ≤5; everything else → propagate
- EMF metrics named (records_in, records_failed, retry_count) + tag dimensions
- Pair-HITL scored ≥14/18 across 6 dims (5 standard + per-PK ordering)
- MR shows 2 commits

⚠ TOP GOTCHAS

Random PK in design

UUID PK destroys ordering. Per-PK ordering needs deterministic PK.

PutRecord loop

1-at-a-time PutRecord throttles fast. PutRecords (≤500) is the right pattern.

No retry budget

Unbounded retry → producer hangs forever. Cap at 5; surface failed records.

Forgot per-PK ordering dim

The 6th HITL dim TODAY is per-PK ordering. Without it, design risk lands in production.

EMF without tag dimensions

Untagged EMF = single global metric, no breakdown. Tag stream + airline.

Hot-shard demo — three PK strategies, observe distribution + ordering

BRIEF

Pair-work. Send 10,000 synthetic cancellation events three ways: (A) static key 'cancel' — guaranteed hot shard. (B) `airline_code:booking_date` — baseline (may skew on burst). (C) `airline_code:booking_date:bucket_hash(pnr) % 8` — recommended. Capture CloudWatch IncomingBytes per shard for all three. Test ordering: send a cancel/refund pair on the SAME PNR using the strict-order path (sequential PutRecord); confirm refund arrives after cancel.

TIME

45 min

LEVEL

Working+

DELIVERABLE

/reviews/m7-hot-shard-evidence.md (per-shard bytes for A and B + ordering test result)

Six steps. Generate 10K → send with PK A → measure → send with PK B → measure → ordering test.

1

Generate 10K synthetic events

```
python3 -m scripts.gen_m7_events \
  --count 10000 --airlines AI,EK,LH,SQ \
  --date-range 2026-05-01 2026-05-10 \
```

EXPECTED

10K events. Mix of airlines + dates. JSON-Lines format.

2

Strategy A — random PK (booking_id)

```
python3 -m scripts.put_to_kinesis \
  --stream accelya-bookings-stream-$LEARNER --region ap-south-1 \
  --pk-from booking_id \
```

EXPECTED

Both shards have ~50% of bytes (random PK ≈ even distribution).

3

Strategy B — deterministic PK (airline+date)

```
python3 -m scripts.put_to_kinesis \
  --stream accelya-bookings-stream-$LEARNER --region ap-south-1 \
  --pk-from airline_code_booking_date \
```

EXPECTED

Both shards have ~50% of bytes (good PK ≈ even distribution).

4

Ordering test — cancel + refund pair

```
# Send a cancel event:
aws kinesis put-record --stream-name accelya-bookings-stream-$LEARNER \
  --partition-key 'AI:2026-05-09' \
  --data "$(echo -n '{"event_time":"2026-05-09T09:00:00","event_id":"CXL-1111"}' | base64) \
```

EXPECTED

Both records on the same shard (same PK). SequenceNumber refund > SequenceNumber cancel.

5

Read back + verify ordering

```
SHARD_ID=$(aws kinesis describe-stream --stream-name accelya-bookings-stream-$LEARNER \
  --region ap-south-1 \
  --query "StreamDescription.Shards[?HashKeyRange.StartingHashKey=<'AI:2026-05-09' |
```

EXPECTED

Two records returned. Sequence numbers strictly increasing. Cancel before refund.

6

Document + commit

```
# /reviews/m7-hot-shard-evidence.md captures:
# ## Strategy A (random PK)
#   Shard 0 bytes: <X>; Shard 1 bytes: <Y>; ratio: <Z>
#   Ordering: random — cancel/refund order not guaranteed
# ## Strategy B (airline+date PK)
#   Shard 0 bytes: <X>; Shard 1 bytes: <Y>; ratio: <Z>
#   Ordering: per-PK preserved (cancel → refund order verified)
```

EXPECTED

MR shows 3 commits.

Success criteria & common gotchas

✓ YOU ARE READY IF...

- Strategy A and Strategy B both run; per-shard IncomingBytes captured
- Both strategies show even distribution (≤55:45 split)
- Ordering test: cancel + refund land on SAME shard with refund SeqNum > cancel SeqNum
- `/reviews/m7-hot-shard-evidence.md` captures both strategies + ordering result + recommendation
- MR shows 3 commits; pair partner tagged

⚠ TOP GOTCHAS

CW metrics lag

Per-shard metrics take 1-3 min to appear. Wait before querying.

Hash collision interpretation

If both PKs hash to the same shard, ordering still works. Use `describe-stream HashKeyRange` to verify which shard.

Forgetting `--shard-iterator-type`

Default is `TRIM_HORIZON`; for live testing use `LATEST`.

Putting test records on wrong PK

If you typo the PK, ordering test is meaningless. Quote it carefully in the CLI.

Reading too late

`TRIM_HORIZON` resets to start; you'll see all 10K records. Use `AT_SEQUENCE_NUMBER` for targeted reads.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-7 streaming mistakes. Cleanup that preserves the stream + DDB table (Day 8 reuses both). Day-7 in numbers. Bridge to Day 8 — Cascade-authors the producer + consumer Lambdas. STATE CONTEXT block (.windsurfrules → v5).

Five Day-7 streaming mistakes — and what to do instead

01

PK that creates hot shard

MISTAKE PK = static string, low-cardinality (airline_code alone), or correlated-with-time (timestamp). All shards end up imbalanced.

DO INSTEAD

PK = composite (airline+date). Cardinality + distribution + ordering all met. Concept 3 hot-shard math.

02

Producer without retry

MISTAKE Producer that doesn't catch ProvisionedThroughputExceededException. First throttle = lost data.

DO INSTEAD

Backoff + jitter ≤ 5 retries. Re-PutRecords just failed. EMF retry_count metric.

03

No IteratorAge alarm

MISTAKE Consumer falls behind silently. Operators discover at on-call time when downstream queries return stale data.

DO INSTEAD

CloudWatch alarm on IteratorAge > 60s. Day 10 wires for production; today the design names the metric.

04

ReportBatchItemFailures off

MISTAKE ESM created without --function-response-types. Lambda returns failures but ESM ignores them. SQS-style trap from Day 4.

DO INSTEAD

Always pass --function-response-types ReportBatchItemFailures to create-event-source-mapping. Verify with list-event-source-mappings.

05

Consumer logic in producer

MISTAKE Producer does enrichment, validation, persistence. Consumer is a thin pass-through. Reverses the partial-batch contract — producer failures become much more expensive.

DO INSTEAD

Producer ONLY validates envelope + PutRecords. Consumer handles enrichment, idempotency, persistence. Producer failures must be cheap to retry.

End-of-module cleanup

CLEANUP CHECKLIST

- Branch pushed: module-7-<learner> with 3 commits
- MR open; pair partner tagged
- Stream accelya-bookings-stream-<learner> ACTIVE — DO NOT DELETE (Day 8 consumes from it)
- DynamoDB stream_seen_keys_<learner> ACTIVE — DO NOT DELETE (Day 8 idempotency)
- IAM role accelya-stream-role-<learner> preserved
- Lambda count cumulative: still 6 of 10 (no new Lambdas today; design-only)
- Bedrock invocations: ~10 of 50 (Cascade-led design + provision script)
- Synthetic 10K test events sent (Lab 3) — they decay from stream after 24h retention

```
# Verify nothing was accidentally deleted:
aws kinesis describe-stream --stream-name accelya-bookings-stream-$LEARNER \
  --region ap-south-1 --query 'StreamDescription.StreamStatus'
aws dynamodb describe-table --table-name stream_seen_keys_$LEARNER \
  --region ap-south-1 --query 'Table.TableStatus'

# Lambda count check (target: still 6):
aws lambda list-functions --region ap-south-1 \
  --query "length(Functions[?contains(FunctionName,'$LEARNER')])"

# Bedrock budget (3 metrics):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics --namespace AWS/Bedrock \
    --metric-name $METRIC --dimensions Name=ModelId,Value=$PROFILE \
    --start-time $(date -u -d 'today 00:00' +%FT%T) \
    --end-time $(date -u +%FT%T) --period 3600 \
    --statistics Sum --region ap-south-1
done
```

Trainer note: stream + idempotency table preserved for Day 8. Lambda count 6 of 10. 4 left for D8 (producer + consumer = +2) + D9 (inference wrapper = +1) + D11 (1 spare).

What you actually shipped

1

STREAM (2 shards)

accelya-bookings-stream-<learner>.
ACTIVE. 24h retention. PK strategy
locked.

0

NEW LAMBDA

Design-heavy day. Day 8 builds
producer + THIN consumer +
bedrock-enrich (count → 9 of 10).

10K

EVENTS TESTED

Two PK strategies. Distribution +
ordering proven via CW metrics.

10

PROMPT LAYERS

+ STREAM CONTEXT slot.
v10_stream.txt committed. Day 8
inherits.

Day 8 — Agentic Real-Time Processing (Kinesis consumer + state machine)

Today designed the stream. Tomorrow Cascade authors the producer + THIN consumer + bedrock-enrich Lambda + the inference-pipeline state machine. Consumer is THIN (claim → StartExecution → ACK); Bedrock + persist live in the SFN state machine. Sampled Bedrock enrichment (every Nth event, not all — costs!). STATE CONTEXT block (.windsurfrules → v5).

PRODUCER + THIN CONSUMER + BEDROCK-ENRICH

+3 Lambdas (count → 9 of 10)

Producer reads events + PutRecords. THIN consumer claims + StartExecutions on the inference-pipeline state machine + ACKs. bedrock-enrich Lambda is the SFN Task that does sampled Bedrock InvokeModel + persist.

11TH PROMPT LAYER

STATE CONTEXT for streams joins the 10

Shard, sequence_number, partial-batch retry rules, sampling cadence, deterministic SFN exec naming, THIN consumer rule. Cascade scaffolds with this context loaded.

SANDBOX BUDGET

Day 8 uses +3 Lambdas + ~15 Bedrock invocations (sampled enrichment)

Cumulative after Day 8: 9 of 10 Lambdas; ~38 of 50 Bedrock invocations. D9 inference wrapper (+1 Lambda → cap reached); D10 observability (+0); D11 ALARM-based rollback (+0).

Day 8 branch: module-8-<learner>. Inherits .windsurfrules v4 (with STREAM CONTEXT). Bring your stream + DDB table.

Stream provisioned. Contracts ratified.

STREAM + IAM + DDB

2-shard Kinesis stream + per-learner IAM + stream_seen_keys table. ACTIVE. Day 8 consumes.

CONTRACTS

Producer + consumer JSON contracts. PK locked: airline+date. Idempotency on sequence_number. ReportBatchItemFailures critical.

.WINDSURFRULES v4

STREAM CONTEXT block added. Cascade now reads stream config before scaffolding any producer/consumer.

Day 8 starts at 09:00. Bring your stream + DDB table. Producer + consumer Lambdas enter; the STATE CONTEXT block (.windsurfrules → v5) joins the .windsurfrules v4 you used today.